

SUPPORT DE COURS COMPLET

Technologie Blockchain

16h CM · 8h TP · 24h

Dr. Zakaria Sawadogo

École Polytechnique de Ouagadougou



Sommaire

Syllabus

1. Chapitre 1 — Introduction : principes fondamentaux de la blockchain
2. Chapitre 2 — Structures de données : chaînage cryptographique et arbres de Merkle
3. Chapitre 3 — Mécanismes de consensus
4. Chapitre 4 — Smart contracts et Ethereum
5. Chapitre 5 — Sécurité et vulnérabilités des smart contracts
6. Chapitre 6 — Applications, enjeux et limites

Technologie Blockchain

Volume horaire : 16h CM · 8h TP (24h)

Objectifs pédagogiques

- Comprendre les principes fondamentaux d'une blockchain (structure de données, chaînage cryptographique, décentralisation).
- Comprendre les principaux mécanismes de consensus (preuve de travail, preuve d'enjeu, consensus byzantin) et leurs compromis.
- Comprendre le fonctionnement des smart contracts et leurs risques de sécurité spécifiques.
- Développer un regard critique sur les usages, limites et enjeux réglementaires de la technologie blockchain.

Prérequis

Ce module s'appuie fortement sur les notions du module *Cryptographie* (fonctions de hachage, signatures numériques, arbres de Merkle) : il est recommandé de suivre ce dernier en amont ou en parallèle.

Plan de séances

Cours magistraux (16h)

Séance	Chapitre	Durée
1	Introduction : principes fondamentaux de la blockchain	2h
2	Structures de données : chaînage cryptographique et arbres de Merkle	3h
3	Mécanismes de consensus	3h
4	Smart contracts et Ethereum	3h
5	Sécurité et vulnérabilités des smart contracts	2h
6	Applications, enjeux et limites	3h

Travaux pratiques (8h)

Séance	Sujet	Durée
1	Implémentation d'une mini-blockchain en Python	2h

Séance	Sujet	Durée
2	Simulation d'un consensus par preuve de travail	2h
3	Développement d'un smart contract simple (Solidity)	2h
4	Audit de vulnérabilités d'un smart contract	2h

Évaluation

- TP notés : 40 %
- Exposé sur un cas d'usage ou une faille blockchain réelle : 20 %
- Examen final : 40 %

Environnement technique

Python pour le TP1/TP2 ; Remix IDE (en ligne, environnement de test local) et Solidity pour les TP3/TP4 — aucune manipulation sur un réseau principal (mainnet) réel ni de fonds réels dans le cadre de ce cours.

Bibliographie

- A. Antonopoulos, *Mastering Bitcoin*.
- A. Antonopoulos, G. Wood, *Mastering Ethereum*.
- S. Nakamoto, *Bitcoin: A Peer-to-Peer Electronic Cash System* (2008).
- ConsenSys, *Smart Contract Best Practices*.

Chapitre 1 — Introduction : principes fondamentaux de la blockchain

1. Définition

Une blockchain est un registre distribué, répliqué sur de nombreux nœuds d'un réseau pair-à-pair, structuré en blocs chaînés cryptographiquement, dont le contenu passé est rendu extrêmement difficile à modifier une fois validé, sans nécessiter d'autorité centrale unique de confiance.

Cette définition volontairement générale mérite d'être décomposée en ses ingrédients constitutifs, chacun étant étudié en détail dans un chapitre ultérieur de ce module :

- un **registre** (*ledger*) : une liste ordonnée et append-only d'enregistrements (les transactions) ;
- **distribué** : ce registre n'existe pas en un exemplaire unique conservé par une entité, mais est répliqué intégralement (ou partiellement, pour les clients légers) chez de nombreux participants indépendants ;
- **pair-à-pair** : les participants communiquent directement entre eux, sans serveur central obligatoire, par un protocole de propagation de type *gossip* ;
- **blocs chaînés cryptographiquement** : les enregistrements sont regroupés en blocs, et chaque bloc référence cryptographiquement le précédent (chapitre 2) ;
- **difficile à modifier une fois validé** : propriété obtenue par la combinaison du chaînage cryptographique et d'un mécanisme de consensus coûteux à tromper (chapitre 3) ;
- **sans autorité centrale unique** : la validation des blocs est distribuée entre de nombreux participants suivant une règle algorithmique connue de tous, et non décidée par un tiers de confiance unique.

Il est important de noter dès à présent qu'aucun de ces ingrédients n'est, pris isolément, une nouveauté informatique : les fonctions de hachage, les listes chaînées, les systèmes répliqués et les protocoles de consensus distribué existaient tous avant 2008. L'apport de Bitcoin est de les **combiner** de façon à obtenir, pour la première fois, un système monétaire numérique fonctionnant sans tiers de confiance central — une synthèse d'ingénierie plus qu'une invention cryptographique isolée.

Registre centralisé, registre partagé, registre distribué

Pour bien situer l'apport de la blockchain, il est utile de comparer trois architectures de registre :

Architecture	Qui détient la vérité ?	Exemple
Registre centralisé	une entité unique (une banque tient les comptes de ses clients)	système bancaire classique, base de données d'entreprise
Registre partagé (répliqué, sans consensus décentralisé)	plusieurs copies existent, mais une entité reste maîtresse des écritures (réplication maître-esclave)	grappe de bases de données classiques répliquées pour la disponibilité

Architecture	Qui détient la vérité ?	Exemple
Registre distribué avec consensus décentralisé (blockchain)	aucune entité unique ; l'accord se fait par un protocole algorithmique entre pairs mutuellement méfiants	Bitcoin, Ethereum

La différence essentielle entre les deuxième et troisième lignes n'est donc pas la simple réplication (qui existe depuis longtemps dans les systèmes distribués classiques), mais la capacité à faire converger vers un état unique un ensemble de participants qui ne se font *pas confiance* les uns envers les autres et qui peuvent inclure des acteurs malveillants — un problème nettement plus difficile que la simple tolérance aux pannes.

2. Genèse : Bitcoin (2008-2009)

Publié sous le pseudonyme Satoshi Nakamoto en 2008, Bitcoin est la première application combinant, de façon inédite, des briques cryptographiques déjà connues (signatures numériques, fonctions de hachage) avec un mécanisme de consensus décentralisé (preuve de travail, chapitre 3) pour résoudre le **problème de la double dépense** sans tiers de confiance central : comment empêcher qu'une même unité de monnaie numérique soit dépensée deux fois, sans banque centrale ?

Le problème de la double dépense

Dans un système de monnaie purement numérique (une suite de bits), rien n'empêche *a priori* un utilisateur de copier cette suite de bits et de la transmettre deux fois à deux bénéficiaires différents — contrairement à un billet physique, qui ne peut se trouver, à un instant donné, que dans une seule main. Avant Bitcoin, la solution standard à ce problème consistait à faire confiance à un tiers centralisé (une banque, un opérateur de monnaie électronique) qui tient un registre unique des soldes et refuse toute transaction qui dépenserait deux fois le même montant. L'apport conceptuel de Bitcoin est de montrer qu'un réseau de participants mutuellement méfiants, sans tiers central, peut néanmoins se mettre d'accord sur un **ordre unique** des transactions — et donc détecter et rejeter toute tentative de double dépense — à condition d'accepter un mécanisme de consensus coûteux (la preuve de travail) qui rend économiquement irrationnelle la falsification de cet ordre.

Des idées antérieures à la synthèse de 2008

Plusieurs briques utilisées par Bitcoin existaient déjà dans la littérature académique avant 2008 : les fonctions de hachage cryptographiques et les arbres de Merkle (Ralph Merkle, fin des années 1970), les systèmes de preuve de travail conçus initialement contre le spam (*Hashcash*, fin des années 1990), et diverses propositions de monnaie numérique décentralisée qui n'avaient pas rencontré d'adoption large. Le mérite de la proposition de Satoshi Nakamoto est d'avoir assemblé ces éléments en un système cohérent, complet et effectivement déployé, avec une incitation économique (la récompense de minage) alignant l'intérêt individuel des participants sur la sécurité collective du réseau.

3. Propriétés recherchées

Propriété	Signification
Décentralisation	aucune entité unique ne contrôle le registre ; il est répliqué et validé par de nombreux participants indépendants
Immutabilité (pratique)	modifier un bloc déjà validé et suffisamment confirmé nécessiterait de refaire le travail de validation de tous les blocs suivants, un coût prohibitif au-delà d'une certaine profondeur
Transparence	(pour les blockchains publiques) l'historique complet des transactions est consultable par tous
Résistance à la censure	aucune autorité unique ne peut empêcher une transaction valide d'être incluse, dans un réseau suffisamment décentralisé

Ces propriétés sont des objectifs de conception, pas des garanties absolues : elles dépendent fortement du degré réel de décentralisation du réseau (chapitre 3) et de la sécurité des implémentations (chapitre 5).

Décentralisation : trois dimensions distinctes

Le terme « décentralisation » est souvent employé de façon floue. Il est utile de le décomposer en (au moins) trois dimensions indépendantes, un même réseau pouvant être décentralisé selon l'une et centralisé selon une autre :

- **décentralisation architecturale** : combien de machines physiques distinctes composent le réseau, et où sont-elles situées ?
- **décentralisation politique** : combien d'individus ou d'organisations distincts contrôlent effectivement ces machines (un même opérateur peut faire tourner des milliers de nœuds) ?
- **décentralisation logique** : l'état et les règles du système forment-ils un ensemble monolithique unique, ou pourrait-il en principe être scindé en deux moitiés indépendantes sans perte de sens (un carnet d'adresses réparti est décentralisé de façon logique ; un registre de comptes bancaires cohérent ne l'est pas) ?

Un exemple pédagogique : un réseau comptant dix mille nœuds répartis dans le monde entier (forte décentralisation architecturale) mais dont 90 % de la puissance de minage appartient à trois entités minières (faible décentralisation politique) offre, en pratique, un niveau de résistance à la censure et de sécurité contre une attaque à 51 % nettement inférieur à ce que suggère le seul décompte des nœuds.

4. Blockchain publique, privée, à permission

Type	Accès en écriture	Accès en lecture	Exemple
Publique (permissionless)	ouvert à tous	ouvert à tous	Bitcoin, Ethereum

Type	Accès en écriture	Accès en lecture	Exemple
À permission (consortium)	limité à des participants identifiés	selon règles définies	réseaux inter-bancaires, chaînes logistiques inter-entreprises
Privée	contrôlé par une seule organisation	souvent restreint	usage interne d'entreprise, proche d'une base de données répliquée classique

Une confusion fréquente : une blockchain « privée » ou « à permission » perd une grande partie des propriétés de décentralisation et de résistance à la censure d'une blockchain publique — le choix du type doit être justifié par le besoin réel, pas par un effet de mode.

Un exemple pédagogique permet d'illustrer ce choix. Imaginons un consortium fictif de cinq entreprises de logistique d'une même région souhaitant partager un registre commun de suivi de conteneurs : les cinq entreprises se connaissent, ont des intérêts partiellement alignés (fluidifier les échanges) et partiellement divergents (chacune souhaite éviter d'être accusée à tort d'un retard). Une blockchain à permission, où chacune des cinq organisations exploite un nœud validateur identifié, répond bien à ce besoin : elle élimine la nécessité de faire confiance à l'infrastructure d'une seule des cinq entreprises, sans pour autant nécessiter l'anonymat et la résistance à la censure d'un réseau ouvert à des inconnus comme Bitcoin. À l'inverse, un cas d'usage où n'importe quel individu dans le monde doit pouvoir participer sans autorisation préalable (un système de paiement mondial résistant à la censure d'un État) requiert par construction une blockchain publique.

5. Anatomie d'un bloc (aperçu)

Chaque bloc contient typiquement : un en-tête (empreinte du bloc précédent, horodatage, racine de Merkle des transactions du bloc — chapitre 2, un nonce pour le mécanisme de consensus), et le corps (liste des transactions incluses).

Structure schématique d'un en-tête de bloc

Le tableau suivant détaille, à titre pédagogique, les champs typiquement présents dans l'en-tête d'un bloc de type Bitcoin — les blockchains à smart contracts (chapitre 4) ajoutent des champs supplémentaires (racine de l'état global, notamment) :

Champ	Rôle
Empreinte du bloc précédent	assure le chaînage cryptographique (chapitre 2)
Horodatage (<i>timestamp</i>)	date approximative de création du bloc, utilisée notamment pour ajuster la difficulté du consensus
Racine de Merkle	résume l'ensemble des transactions du bloc en une seule empreinte (chapitre 2)
Nonce	valeur ajustée par le mineur pour satisfaire la condition de difficulté du consensus (chapitre 3)

Champ	Rôle
Numéro de version / difficulté	paramètres de protocole utilisés par les nœuds pour valider le bloc

Un point pédagogique important : l'en-tête d'un bloc est de taille fixe et petite (quelques dizaines d'octets), indépendamment du nombre de transactions qu'il résume — c'est précisément ce que permet l'usage de la racine de Merkle plutôt que la liste complète des transactions (chapitre 2). Cette propriété est ce qui rend possible la vérification par des clients légers ne téléchargeant que les en-têtes de blocs.

6. Ce que la blockchain résout, et ce qu'elle ne résout pas

- Elle résout la **cohérence d'un registre partagé sans tiers de confiance unique**, dans un contexte où les participants peuvent être mutuellement méfiants.
- Elle ne résout pas, par elle-même, la **véracité des données saisies** (« garbage in, garbage out ») : une blockchain garantit l'intégrité de ce qui y a été inscrit, pas la véracité de l'information au moment de sa saisie (problème dit de l'« oracle », chapitre 4).
- Elle a un coût réel : stockage répliqué, consommation énergétique pour certains mécanismes de consensus (chapitre 3), latence de confirmation, complexité opérationnelle — d'où l'importance d'évaluer si une base de données traditionnelle ne répondrait pas mieux à un besoin donné (chapitre 6).

Le problème de l'oracle, une première approche

Ce point mérite d'être approfondi dès l'introduction, car il est fréquemment source de malentendus chez les étudiants découvrant la blockchain : une blockchain garantit l'intégrité de ce qui a été inscrit dans le registre (personne ne peut modifier discrètement une transaction déjà validée), mais elle ne peut, par construction, rien garantir sur la véracité de l'information au moment où elle y a été inscrite. Un exemple pédagogique simple : si un entrepôt fictif enregistre sur une blockchain « 500 sacs de céréales reçus le 12 mars », la blockchain garantit qu'aucun acteur ne pourra plus tard prétendre que l'enregistrement disait « 400 sacs » — mais rien ne garantit que 500 sacs ont réellement été livrés ce jour-là plutôt que 300. Ce problème, dit **problème de l'oracle**, est repris et approfondi au chapitre 4 dans le contexte spécifique des smart contracts, où il devient particulièrement critique lorsque des sommes d'argent sont automatiquement transférées sur la base d'une donnée externe.

Comparaison synthétique avec une base de données traditionnelle

Le tableau suivant résume, à titre pédagogique, les différences de compromis entre une blockchain publique et une base de données centralisée classique, afin de préparer la grille de décision développée au chapitre 6 :

Critère	Base de données centralisée	Blockchain publique
Débit de transactions	très élevé (dépend uniquement du matériel)	limité par le mécanisme de consensus (chapitre 3)

Critère	Base de données centralisée	Blockchain publique
Tiers de confiance nécessaire	oui (l'opérateur de la base)	non, si le réseau est suffisamment décentralisé
Coût de modification a posteriori	faible (une simple requête UPDATE)	très élevé au-delà de quelques confirmations
Résistance à la censure	dépend entièrement de l'opérateur	forte, si le réseau est suffisamment décentralisé
Coût opérationnel	maîtrisé, proportionnel à l'usage	réplication systématique chez tous les nœuds, potentiellement coûteuse en énergie (PoW)

À retenir

- La blockchain combine des primitives cryptographiques connues (fonctions de hachage, signatures numériques) avec un mécanisme de consensus décentralisé pour résoudre le problème de la double dépense sans autorité centrale — une synthèse d'ingénierie plutôt qu'une rupture cryptographique isolée.
- Les propriétés de décentralisation, immutabilité et résistance à la censure sont des objectifs de conception dont le degré réel dépend de l'architecture choisie ; la décentralisation elle-même se décompose en dimensions architecturale, politique et logique qui peuvent diverger.
- Le choix entre blockchain publique, à permission ou privée doit être guidé par le besoin réel (nombre et confiance mutuelle des participants, nécessité de résistance à la censure), pas par un effet de mode.
- Le problème de l'oracle rappelle qu'une blockchain garantit l'intégrité de ce qui y est inscrit, jamais la véracité de l'information au moment de sa saisie — un point central repris au chapitre 4.
- Une blockchain n'est pas une solution universelle : son usage doit être justifié par un besoin réel de décentralisation, pas adopté par principe, et se compare toujours à l'alternative d'une base de données traditionnelle.

Chapitre 2 — Structures de données : chaînage cryptographique et arbres de Merkle

1. Le chaînage cryptographique des blocs

Chaque bloc contient, dans son en-tête, l'**empreinte de hachage du bloc précédent**. Cette construction crée une dépendance en cascade : modifier le contenu d'un bloc ancien change son empreinte, ce qui invalide l'empreinte enregistrée dans le bloc suivant, et ainsi de suite jusqu'au sommet de la chaîne. Un attaquant souhaitant altérer un bloc ancien devrait donc recalculer (et, selon le mécanisme de consensus, refaire valider) tous les blocs suivants plus rapidement que le reste du réseau honnête — d'où le terme d'immuabilité « pratique » plutôt qu'absolue, et la recommandation d'attendre un nombre suffisant de confirmations avant de considérer une transaction comme définitive.

Illustration pédagogique du chaînage

Le fragment Python suivant (purement pédagogique, très simplifié par rapport à une implémentation réelle) illustre le principe du chaînage sur une liste de blocs minimalistes, afin de rendre concrète l'idée de « dépendance en cascade » :

```
import hashlib

def empreinte(bloc: dict) -> str:
    """Calcule l'empreinte d'un bloc à partir de ses champs (exemple pédagogique)."""
    contenu = f"{bloc['index']}{bloc['precedent']}{bloc['donnees']}{bloc['nonce']}"
    return hashlib.sha256(contenu.encode()).hexdigest()

# Construction d'une mini-chaîne de 3 blocs
genese = {"index": 0, "precedent": "0" * 64, "donnees": "bloc genese", "nonce": 0}
genese["empreinte"] = empreinte(genese)

bloc1 = {"index": 1, "precedent": genese["empreinte"], "donnees": "Alice paie Bob", "nonce": 0}
bloc1["empreinte"] = empreinte(bloc1)

bloc2 = {"index": 2, "precedent": bloc1["empreinte"], "donnees": "Bob paie Chloé", "nonce": 0}
bloc2["empreinte"] = empreinte(bloc2)

# Si l'on modifie les données du bloc1 après coup...
bloc1["donnees"] = "Alice paie Eve" # falsification tentée
# ...son empreinte change, ce qui invalide le champ "precedent" enregistré dans bloc2 :
assert empreinte(bloc1) != bloc1["empreinte"] # l'empreinte stockée ne correspond p
assert bloc2["precedent"] != empreinte(bloc1) # la cascade de rupture se propage ve
```

Cet exemple simplifié omet volontairement l'arbre de Merkle (une seule donnée par bloc) et le mécanisme de consensus (le `nonce` n'est pas ajusté pour satisfaire une difficulté) : il vise uniquement à rendre visible, en quelques lignes, la propriété de dépendance en cascade au cœur de l'immuabilité pratique.

2. Arbre de Merkle

Un arbre de Merkle organise un ensemble de données (les transactions d'un bloc) de façon à pouvoir résumer l'ensemble par une seule empreinte (la **racine de Merkle**) tout en permettant de prouver efficacement qu'une donnée particulière fait bien partie de l'ensemble, sans avoir à transmettre l'ensemble complet.

Construction

1. Chaque transaction est hachée individuellement (feuilles de l'arbre).
2. Les empreintes sont regroupées deux par deux et hachées ensemble pour former le niveau supérieur.
3. Ce processus est répété jusqu'à obtenir une seule empreinte : la racine de Merkle, stockée dans l'en-tête du bloc.

Lorsque le nombre de feuilles à un niveau donné est impair, une convention doit être adoptée pour former les paires (par exemple dupliquer la dernière feuille) ; ce détail d'implémentation, anodin en apparence, a par le passé été source de subtilités de sécurité dans certaines implémentations réelles (risque de confusion entre deux ensembles de transactions différents produisant, par construction naïve, la même racine) — une illustration du principe général selon lequel les détails d'implémentation d'une primitive cryptographique ont une importance de sécurité directe, et non purement cosmétique.

Exemple pédagogique de construction (Python)

```
import hashlib

def h(x: bytes) -> bytes:
    return hashlib.sha256(x).digest()

def racine_merkle(transactions: list[bytes]) -> bytes:
    """Construit itérativement la racine de Merkle d'une liste de transactions
    (implémentation pédagogique, sans les optimisations d'une bibliothèque réelle)."""
    niveau = [h(tx) for tx in transactions]          # feuilles : hachage de chaque tra
    if not niveau:
        return h(b"")
    while len(niveau) > 1:
        if len(niveau) % 2 == 1:
            niveau.append(niveau[-1])                # convention : dupliquer la derni
        niveau = [h(niveau[i] + niveau[i + 1]) for i in range(0, len(niveau), 2)]
    return niveau[0]

transactions = [b"Alice->Bob:10", b"Bob->Chloe:4", b"Chloe->Alice:1", b"Alice->David:2"]
print(racine_merkle(transactions).hex())
```

Preuve de Merkle (Merkle proof)

Pour prouver qu'une transaction T appartient à un bloc, il suffit de fournir T et la liste des empreintes « sœurs » sur le chemin vers la racine (de l'ordre de $\log_2(n)$ empreintes pour n transactions), plutôt que l'ensemble des n transactions. Cette propriété est essentielle pour les **clients légers (SPV —**

Simplified Payment Verification), qui vérifient qu'une transaction est incluse dans un bloc sans télécharger l'intégralité de la blockchain.

L'efficacité de cette preuve est ce qui rend la structure de Merkle si utile en pratique : pour un bloc contenant, par exemple, 2 048 transactions (2^{11}), une preuve d'inclusion ne nécessite que 11 empreintes intermédiaires (plus la racine déjà connue), soit un volume de données de l'ordre de quelques centaines d'octets, indépendamment du fait que le bloc entier pèse potentiellement plusieurs mégaoctets. C'est un exemple caractéristique de structure de données offrant une **preuve de taille logarithmique** pour une propriété portant sur un ensemble de taille linéaire — un compromis extrêmement favorable, comparé à la transmission de l'ensemble complet des transactions qu'exigerait une simple liste non structurée.

Exemple pédagogique de vérification d'une preuve

```
def verifier_preuve(feuille: bytes, preuve: list[tuple[bytes, str]], racine_attendue: bytes) -> bool:
    """preuve est une liste de tuples (empreinte_sœur, position) où position vaut
    'gauche' ou 'droite', indiquant de quel côté placer l'empreinte sœur (exemple pédagogique)"""
    courant = h(feuille)
    for empreinte_sœur, position in preuve:
        if position == "gauche":
            courant = h(empreinte_sœur + courant)
        else:
            courant = h(courant + empreinte_sœur)
    return courant == racine_attendue
```

Ce court exemple illustre un point pédagogique important : la vérification d'une preuve de Merkle ne nécessite que la fonction de hachage et la preuve elle-même — aucune donnée de l'arbre entier n'est requise, ce qui permet à un client léger de vérifier une transaction avec des ressources minimales (mémoire, bande passante).

3. Pourquoi cette double structure (chaîne de blocs + arbre par bloc) ?

- Le **chaînage entre blocs** garantit l'intégrité de l'ordre et de l'historique global.
- L'**arbre de Merkle au sein d'un bloc** garantit l'intégrité de l'ensemble des transactions d'un bloc tout en permettant des vérifications partielles efficaces.

Cette architecture illustre directement l'usage des fonctions de hachage (module *Cryptographie*, chapitre 3) comme brique de construction pour des propriétés de plus haut niveau (intégrité globale d'un registre distribué).

Propriétés cryptographiques requises de la fonction de hachage

Cette double structure ne fonctionne correctement que si la fonction de hachage utilisée possède certaines propriétés étudiées en détail dans le module *Cryptographie* :

- **résistance aux collisions** : il doit être calculatoirement infaisable de trouver deux entrées distinctes produisant la même empreinte — sans cela, un attaquant pourrait substituer une

transaction frauduleuse à une transaction légitime tout en conservant la même racine de Merkle ;

- **résistance à la préimage** : il doit être infaisable, à partir d'une empreinte, de retrouver une entrée qui la produit — propriété moins critique ici que pour le stockage de mots de passe, mais toujours pertinente ;
- **effet avalanche** : une modification infime de l'entrée (un seul bit) doit produire une empreinte de sortie complètement différente, sans corrélation apparente avec l'entrée d'origine — c'est cette propriété qui garantit qu'aucune modification, même mineure, d'une transaction ancienne ne peut passer inaperçue dans la racine de Merkle ni dans le chaînage des blocs.

Bitcoin et Ethereum utilisent des fonctions de la famille SHA-2/SHA-3/Keccak, étudiées en détail dans le chapitre correspondant du module *Cryptographie* ; ce chapitre présuppose cette base et se concentre sur l'usage qui en est fait pour structurer un registre distribué.

4. Identifiants et adresses

- Une **transaction** est identifiée par l'empreinte de son propre contenu (le « hash de transaction »).
- Une **adresse** (compte) dérive généralement d'une clé publique (elle-même souvent hachée pour former une adresse plus courte), reliant directement ce chapitre au module *Cryptographie* (signatures numériques, chapitre 5) : chaque transaction est signée par la clé privée correspondant à l'adresse d'origine, et cette signature est vérifiable par tout nœud du réseau.

Modèle UTXO contre modèle de compte

Deux grandes familles de modèles de représentation des soldes coexistent dans les blockchains actuelles, avec des implications directes sur la façon dont les transactions sont structurées et vérifiées :

Modèle	Principe	Exemple
UTXO (<i>Unspent Transaction Output</i>)	chaque transaction consomme un ou plusieurs « jetons » de sortie non dépensés issus de transactions antérieures, et en produit de nouveaux ; le solde d'une adresse est la somme des UTXO qui lui sont associés	Bitcoin
Modèle de compte	chaque adresse possède un solde global directement mis à jour (comme un compte bancaire classique), analogue au champ <code>soldes</code> de l'exemple de smart contract présenté au chapitre 4	Ethereum

Le modèle UTXO offre un parallélisme naturel pour la validation (des UTXO indépendants peuvent être vérifiés séparément) et facilite le respect de la confidentialité (une nouvelle adresse peut être générée pour chaque transaction reçue), mais rend plus complexe la représentation d'un état global mutable — d'où le choix du modèle de compte pour Ethereum, mieux adapté à l'exécution de smart contracts à état persistant (chapitre 4).

5. Variantes et structures alternatives

- **DAG (Directed Acyclic Graph)** : certaines architectures (ex. IOTA) remplacent la chaîne linéaire de blocs par un graphe orienté acyclique de transactions, visant une meilleure scalabilité, au prix d'autres compromis de sécurité et de maturité.
- **État global (state trie)** : les blockchains à smart contracts (Ethereum, chapitre 4) maintiennent, en plus de l'historique des transactions, une structure arborescente représentant l'état courant de tous les comptes et contrats, également authentifiée cryptographiquement (racine d'état incluse dans chaque bloc).

Ethereum va plus loin que le simple couple chaînage + arbre de Merkle des transactions : chaque bloc y contient en réalité **trois racines distinctes** dans son en-tête — la racine de Merkle des transactions du bloc, une racine résumant l'état global de tous les comptes et contrats après exécution du bloc (le *state trie*, techniquement un arbre de Merkle-Patricia, une variante combinant arbre de Merkle et arbre préfixe pour un accès et une mise à jour efficaces), et une racine résumant les reçus d'exécution des transactions. Cette architecture plus riche prépare directement le chapitre 4, où l'état global inclut non seulement des soldes mais aussi le code et les variables persistantes des smart contracts déployés.

Limites de la structure de Merkle simple

Il est utile, à ce stade, de noter une limite pédagogique de l'arbre de Merkle « statique » tel que présenté en section 2 : il est optimisé pour prouver l'appartenance d'un élément à un ensemble **figé** au moment de la construction de l'arbre. Or l'état global d'une blockchain à smart contracts change à chaque bloc (nouveaux soldes, nouvelles variables de contrat). Les arbres de Merkle-Patricia (et plus généralement les *arbres de Merkle persistants* ou *versionnés*) répondent à ce besoin en permettant de dériver efficacement, à chaque mise à jour, une nouvelle racine sans reconstruire l'arbre entier depuis zéro — un raffinement d'ingénierie que ce cours ne détaille pas davantage mais qu'il est utile de savoir identifier pour la suite du module.

À retenir

- Le chaînage cryptographique entre blocs et l'arbre de Merkle au sein d'un bloc sont deux mécanismes complémentaires garantissant respectivement l'intégrité de l'historique global et celle du contenu de chaque bloc.
- Une preuve de Merkle permet de vérifier l'inclusion d'une transaction en un nombre d'étapes logarithmique par rapport au nombre de transactions du bloc, sans télécharger l'intégralité de la blockchain — fondement des clients légers (SPV).
- Le bon fonctionnement de cette architecture dépend de propriétés cryptographiques précises de la fonction de hachage utilisée (résistance aux collisions, effet avalanche), étudiées en détail dans le module *Cryptographie*.
- Le modèle UTXO (Bitcoin) et le modèle de compte (Ethereum) représentent différemment les soldes et ont des implications directes sur le parallélisme de validation et la facilité à représenter un état mutable complexe.
- Les blockchains à smart contracts étendent la structure vue dans ce chapitre avec une racine d'état global (state trie), authentifiant non seulement les transactions mais tout l'état courant du réseau.

Chapitre 3 — Mécanismes de consensus

1. Pourquoi un mécanisme de consensus ?

Dans un réseau décentralisé sans autorité centrale, il faut un mécanisme permettant à des participants potentiellement nombreux, anonymes et parfois malveillants de s'accorder sur un état unique et cohérent du registre (quel bloc suit quel bloc), tout en résistant à des tentatives de manipulation. C'est une instance du problème classique dit **des généraux byzantins** en informatique distribuée.

Le problème des généraux byzantins

Formulé initialement en informatique distribuée classique, ce problème imagine plusieurs généraux d'une armée assiégeant une ville, communiquant uniquement par messagers (dont certains peuvent être interceptés, retardés, ou porter de faux messages), devant néanmoins se coordonner sur une décision commune (attaquer ou battre en retraite) malgré la présence possible de généraux traîtres parmi eux. La difficulté centrale est double : il faut à la fois **s'accorder** sur une décision commune (tous les généraux honnêtes doivent choisir la même action) et le faire **malgré des messages potentiellement falsifiés ou perdus**, sans savoir a priori qui, parmi les participants, est malveillant.

Transposé à une blockchain publique, chaque nœud du réseau joue le rôle d'un général : les « messages » sont les blocs et transactions propagés sur le réseau pair-à-pair, et les « généraux traîtres » sont les nœuds malveillants tentant de faire accepter par le reste du réseau une version falsifiée de l'historique (par exemple pour dépenser deux fois la même unité de monnaie). La différence essentielle avec le problème académique classique est l'échelle : une blockchain publique doit résister à un nombre de participants potentiellement très grand, anonymes, pouvant rejoindre ou quitter le réseau à tout moment (contrairement aux formulations académiques classiques qui supposent un ensemble de participants fixe et connu à l'avance) — c'est cette contrainte supplémentaire qui a motivé des mécanismes de consensus spécifiques comme la preuve de travail.

Vivacité et sûreté : deux propriétés à concilier

Tout protocole de consensus distribué doit, en théorie, satisfaire deux propriétés dont la combinaison est difficile dans un réseau asynchrone et adversarial :

- **sûreté (safety)** : le protocole ne doit jamais faire converger des nœuds honnêtes vers deux états définitifs contradictoires (pas de double dépense acceptée) ;
- **vivacité (liveness)** : le protocole doit continuer à progresser (produire de nouveaux blocs) malgré la présence de nœuds défaillants ou malveillants, et ne pas rester bloqué indéfiniment.

Les différents mécanismes présentés dans ce chapitre opèrent des compromis différents entre ces deux propriétés, sous des hypothèses différentes sur le réseau (synchrone ou non) et sur la proportion de participants malveillants tolérée.

2. Preuve de travail (Proof of Work, PoW)

Principe : pour proposer un nouveau bloc, un participant (« mineur ») doit résoudre un défi calculatoire coûteux — trouver une valeur (le *nonce*) telle que l'empreinte de hachage du bloc satisfasse une condition de difficulté (par exemple, commencer par un certain nombre de zéros). Ce calcul est intentionnellement coûteux à réaliser mais trivial à vérifier par les autres nœuds.

- **Sécurité** : falsifier l'historique nécessiterait de disposer d'une puissance de calcul supérieure à celle du reste du réseau honnête combiné (« attaque des 51 % »), un coût économique dissuasif pour un réseau suffisamment décentralisé et puissant.
- **Limite majeure** : consommation énergétique très importante (Bitcoin), motivant la recherche d'alternatives.

Illustration pédagogique de la preuve de travail

Le fragment Python suivant illustre, de façon très simplifiée, la recherche d'un nonce satisfaisant une condition de difficulté arbitraire (nombre de zéros en début d'empreinte) — un exercice de démonstration, sans commune mesure avec la difficulté réelle du réseau Bitcoin :

```
import hashlib

def miner(donnees_bloc: str, difficulte: int) -> tuple[int, str]:
    """Recherche par force brute un nonce tel que l'empreinte du bloc commence
    par `difficulte` zéros hexadécimaux (exemple pédagogique très simplifié)."""
    prefixe_cible = "0" * difficulte
    nonce = 0
    while True:
        contenu = f"{donnees_bloc}{nonce}".encode()
        empreinte = hashlib.sha256(contenu).hexdigest()
        if empreinte.startswith(prefixe_cible):
            return nonce, empreinte
        nonce += 1

nonce, empreinte = miner("Alice paie Bob 10 unités", difficulte=5)
print(f"Nonce trouvé : {nonce}, empreinte : {empreinte}")
```

Ce court exemple permet d'illustrer deux propriétés essentielles de la preuve de travail : la recherche du nonce (le « minage ») est coûteuse (il faut essayer de nombreuses valeurs avant de trouver une empreinte satisfaisant la condition, le coût moyen croissant exponentiellement avec le paramètre `difficulte`), tandis que la **vérification** d'une solution proposée est quasi instantanée (il suffit de recalculer une seule empreinte). C'est cette asymétrie coût-de-production / coût-de-vérification qui est au cœur de tous les mécanismes de preuve de travail.

Ajustement dynamique de la difficulté

Un aspect souvent sous-estimé : la difficulté n'est pas figée, mais réajustée périodiquement par le protocole (toutes les 2016 blocs pour Bitcoin, soit environ deux semaines compte tenu du temps de bloc cible de dix minutes) afin de maintenir un rythme moyen de production de blocs approximativement constant, indépendamment des variations de la puissance de calcul totale déployée

par les mineurs du réseau. Ce mécanisme d'auto-régulation est essentiel : sans lui, une augmentation de la puissance de calcul totale (par exemple, l'arrivée massive de nouveaux mineurs) accélérerait mécaniquement la production de blocs, avec des conséquences sur la sécurité (moins de temps pour la propagation réseau, augmentant le risque de forks accidentels) et sur l'économie du réseau (rythme d'émission de nouvelles unités monétaires).

Attaque des 51 % : mécanique et limites pratiques

Une attaque dite « des 51 % » consiste, pour un attaquant disposant d'une majorité de la puissance de calcul du réseau, à construire en secret une chaîne alternative plus longue que la chaîne publique (par exemple en omettant une transaction qu'il a lui-même émise), puis à la diffuser d'un coup pour que le réseau, suivant la règle de la chaîne la plus longue, l'adopte à la place de la chaîne légitime — permettant potentiellement une double dépense. Il est important de noter les limites pratiques de cette attaque, même en cas de réussite technique : elle ne permet pas de créer des unités monétaires à partir de rien, ni de voler des fonds d'adresses dont l'attaquant ne détient pas la clé privée (les signatures restent vérifiées normalement, chapitre 2) ; elle permet uniquement de réorganiser l'ordre récent des transactions et, potentiellement, d'annuler ses propres transactions récentes déjà confirmées. Pour un réseau suffisamment décentralisé et disposant d'une puissance de calcul totale élevée, le coût d'acquisition ou de location d'une telle puissance de calcul majoritaire, ainsi que le risque de dévaluer la cryptomonnaie que l'attaquant espère précisément exploiter, constituent des freins économiques dissuasifs — un raisonnement de sécurité économique plutôt que purement cryptographique.

3. Preuve d'enjeu (Proof of Stake, PoS)

Principe : le droit de proposer et de valider un bloc est attribué (souvent par tirage pondéré) en fonction de la quantité de cryptomonnaie qu'un participant a « mise en jeu » (staked), plutôt qu'en fonction de sa puissance de calcul.

- **Sécurité** : un participant malveillant risque de perdre (« slashing ») une partie de sa mise s'il tente de valider des blocs contradictoires ou invalides, ce qui aligne son intérêt économique avec le comportement honnête.
- **Avantage** : consommation énergétique très inférieure à PoW (Ethereum est passé de PoW à PoS en 2022, réduisant sa consommation énergétique de plus de 99 %).
- **Débat en cours** : risque de centralisation accrue au profit des plus gros détenteurs (« les riches deviennent validateurs, les validateurs gagnent des récompenses, les riches s'enrichissent davantage »).

Mécanique simplifiée d'un tirage pondéré par la mise

Le pseudo-code suivant illustre, de façon pédagogique et très simplifiée, l'idée d'un tirage au sort pondéré par la mise, sans reproduire les détails effectifs (souvent plus complexes, impliquant des comités de validateurs et des tours multiples) des implémentations réelles :

```
import random

def choisir_validateur(validateurs: dict[str, float]) -> str:
    """validateurs : dictionnaire {adresse: montant mis en jeu}.
    Retourne l'adresse choisie pour proposer le prochain bloc,
    avec une probabilité proportionnelle à sa mise (exemple pédagogique)."""
    adresses = list(validateurs.keys())
    poids = list(validateurs.values())
    return random.choices(adresses, weights=poids, k=1)[0]

validateurs = {"0xAlice": 320, "0xBob": 80, "0xChloe": 600}
print(choisir_validateur(validateurs))  # Chloé a la probabilité la plus élevée d'être
```

Le slashing : aligner incitations et sécurité

Le mécanisme de *slashing* (confiscation partielle ou totale de la mise) est le pivot économique de la sécurité en preuve d'enjeu. Il cible typiquement deux catégories de comportements fautifs, détectables algorithmiquement et de façon vérifiable par tout le réseau :

- **double signature (equivocation)** : un validateur signe deux blocs concurrents à la même hauteur de chaîne, une preuve cryptographique irréfutable de tentative de fraude (les deux signatures, publiées, suffisent à démontrer la faute) ;
- **inactivité prolongée** : un validateur hors ligne qui ne participe plus à la validation, pénalisé (généralement plus légèrement qu'une double signature) pour maintenir la vivacité du réseau.

Un point pédagogique important à souligner aux étudiants : contrairement à la preuve de travail, où le coût de l'attaque est un coût **externe** au système (l'électricité et le matériel consommés, perdus qu'il y ait fraude ou non), le coût du slashing est un coût **interne** directement prélevé sur l'actif que l'attaquant détenait dans le système — un changement de nature du mécanisme de dissuasion, pas seulement de son intensité.

Variantes et raffinements du PoS

Plusieurs familles de conception existent au sein même de la preuve d'enjeu, chacune répondant différemment au compromis vivacité/sûreté/décentralisation :

- **PoS délégué (DPoS)** : les détenteurs de jetons votent pour un nombre restreint de délégués chargés de valider les blocs en leur nom, améliorant potentiellement le débit de transactions au prix d'une décentralisation politique réduite (peu de délégués effectifs) ;
- **PoS liquide** : les détenteurs peuvent déléguer leur pouvoir de validation à un tiers tout en conservant la propriété de leurs jetons et la possibilité de retirer leur délégation à tout moment ;
- **Comités de validateurs par époque** : plutôt qu'un tirage individuel à chaque bloc, un sous-ensemble de validateurs est désigné pour une période donnée (une « époque »), réduisant la surface d'attaque à un instant donné.

4. Tolérance byzantine pratique (PBFT et variantes)

Utilisée typiquement dans les blockchains à permission (nombre de participants limité et identifié) : les nœuds valident un bloc par un mécanisme de vote explicite, tolérant qu'une fraction des participants (typiquement jusqu'à un tiers) soit défaillante ou malveillante, avec une finalité de validation quasi immédiate (contrairement à PoW où la certitude augmente progressivement avec les confirmations).

Pourquoi la limite d'un tiers ?

Cette limite n'est pas arbitraire : elle découle d'un résultat classique de la théorie de la tolérance aux fautes byzantines, selon lequel un protocole de consensus déterministe dans un réseau partiellement synchrone ne peut garantir simultanément la sûreté et la vivacité que si le nombre de participants défaillants ou malveillants reste strictement inférieur à un tiers du total des participants. Intuitivement, avec exactement un tiers de participants byzantins, un attaquant pourrait, dans le pire des cas, faire pencher artificiellement un vote en se positionnant tantôt d'un côté tantôt de l'autre selon ce qui sert sa fraude, empêchant les nœuds honnêtes de distinguer de façon fiable la majorité légitime. Ce résultat, bien antérieur à la blockchain, explique pourquoi les protocoles BFT classiques (comme PBFT, *Practical Byzantine Fault Tolerance*, proposé à la fin des années 1990) et leurs dérivés utilisés dans les blockchains à permission modernes retiennent systématiquement ce seuil.

Déroulement schématique d'un tour de vote BFT

À titre pédagogique, un protocole de type PBFT se déroule typiquement en plusieurs phases de communication entre validateurs, avant qu'un bloc ne soit considéré comme définitivement validé :

1. **Proposition** : un validateur désigné (« le leader » du tour courant) diffuse un bloc candidat à l'ensemble des validateurs.
2. **Pré-vote** : chaque validateur diffuse aux autres son accord (ou désaccord) sur le bloc proposé, après vérification de sa validité.
3. **Vote de validation** : si un validateur observe qu'une majorité qualifiée (généralement plus des deux tiers) a pré-voté en faveur du même bloc, il diffuse à son tour un vote de validation.
4. **Finalisation** : dès qu'un validateur observe qu'une majorité qualifiée de votes de validation a été atteinte pour ce bloc, il le considère comme définitivement finalisé — sans possibilité de réorganisation ultérieure, contrairement à la finalité probabiliste de la preuve de travail.

Ce déroulement en plusieurs tours de communication explique pourquoi les protocoles BFT classiques passent difficilement à l'échelle au-delà de quelques dizaines ou centaines de validateurs (le volume de messages échangés croît rapidement avec le nombre de participants), ce qui justifie leur usage privilégié dans des contextes à permission où le nombre de validateurs reste maîtrisé, plutôt que dans un réseau public ouvert à un nombre non borné de participants anonymes.

5. Comparaison synthétique

Mécanisme	Coût de sécurité	Consommation énergétique	Finalité	Exemple
PoW	puissance de calcul	très élevée	probabiliste (croît avec les confirmations)	Bitcoin

Mécanisme	Coût de sécurité	Consommation énergétique	Finalité	Exemple
PoS	capital économique en jeu	faible	souvent quasi immédiate selon l'implémentation	Ethereum (depuis 2022)
BFT	identité/réputation des validateurs	faible	immédiate	réseaux à permission, certaines blockchains d'entreprise

6. Fork et résolution de conflits

Lorsque deux blocs valides sont proposés presque simultanément (situation normale dans un réseau distribué), une **fourche (fork)** temporaire apparaît ; les nœuds convergent généralement vers la chaîne la plus longue (PoW) ou celle validée par le plus de participants (PoS) selon la règle de choix définie par le protocole. Un **hard fork** délibéré (changement de règles incompatible) peut aussi survenir lors d'une mise à jour majeure ou d'un désaccord de gouvernance dans la communauté.

Fork accidentel contre fork délibéré

Il est pédagogiquement utile de distinguer clairement deux situations souvent confondues par les étudiants découvrant le sujet :

- le **fork accidentel (temporaire)** est un phénomène normal et attendu du fonctionnement d'un réseau distribué : deux mineurs ou validateurs trouvent, à quelques secondes d'intervalle, un bloc valide chacun sur leur propre vue de la chaîne ; le réseau se scinde brièvement en deux groupes de nœuds ayant reçu l'un ou l'autre bloc en premier, avant de converger automatiquement vers une seule chaîne dès que le bloc suivant est ajouté à l'une des deux branches (règle de la chaîne la plus longue, ou son équivalent en PoS) ; les transactions du bloc écarté (« orphelin ») retournent en attente et seront généralement incluses dans un bloc ultérieur ;
- le **hard fork délibéré** est un changement volontaire et incompatible des règles du protocole (par exemple, modifier la limite de taille des blocs, ou corriger une faille de sécurité découverte après coup) : les nœuds n'ayant pas mis à jour leur logiciel continuent de suivre les anciennes règles, ce qui peut faire naître **deux chaînes durablement distinctes** si une partie de la communauté refuse la mise à jour — un cas de figure qui s'est déjà produit dans l'histoire de plusieurs blockchains publiques majeures, y compris à la suite de désaccords de gouvernance profonds sur la marche à suivre après un incident de sécurité important. Un **soft fork**, par contraste, restreint les règles de validité de façon rétrocompatible (un bloc valide selon les nouvelles règles reste valide selon les anciennes), ce qui limite le risque de scission durable du réseau.

À retenir

- Le mécanisme de consensus répond à un problème classique d'informatique distribuée (le problème des généraux byzantins) : accord dans un réseau potentiellement malveillant, sans autorité centrale, en conciliant sûreté (pas de décisions contradictoires) et vivacité (progression continue).
- PoW sécurise par un coût calculatoire externe au système (électricité, matériel) mais consomme beaucoup d'énergie, avec un mécanisme d'ajustement dynamique de la difficulté ; PoS sécurise par un risque économique interne et direct (slashing d'une mise déjà engagée dans le système) avec une empreinte énergétique bien moindre.
- La tolérance byzantine pratique (BFT) offre une finalité de validation immédiate et déterministe, au prix d'un passage à l'échelle plus difficile au-delà de quelques centaines de validateurs, ce qui la réserve typiquement aux réseaux à permission.
- Un fork accidentel (temporaire, résolu automatiquement par la règle de choix de chaîne) doit être distingué d'un hard fork délibéré (changement de règles incompatible, pouvant scinder durablement une communauté en deux chaînes distinctes).
- Le choix du mécanisme de consensus détermine directement les propriétés de sécurité, de décentralisation réelle et de coût du réseau — un compromis, pas une hiérarchie absolue de qualité, à mettre en regard du trilemme scalabilité/sécurité/décentralisation développé au chapitre 6.

Chapitre 4 — Smart contracts et Ethereum

1. Qu'est-ce qu'un smart contract ?

Un smart contract (« contrat intelligent ») est un programme informatique déployé sur une blockchain, dont le code et l'état s'exécutent de façon déterministe et vérifiable par l'ensemble des nœuds du réseau, sans possibilité de modification unilatérale une fois déployé (sauf mécanisme d'évolutivité explicitement prévu). Le terme « contrat » est trompeur au sens juridique : il s'agit avant tout d'un **programme auto-exécutable**, dont la portée juridique effective dépend du contexte réglementaire.

Origine du concept

L'idée d'un « contrat intelligent » précède de plusieurs décennies son implémentation sur blockchain : le terme est attribué au chercheur Nick Szabo, qui décrivait dans les années 1990 l'idée d'encoder les clauses d'un contrat dans un logiciel capable de les exécuter automatiquement, avant même l'existence d'une infrastructure décentralisée capable de l'exécuter de façon fiable et vérifiable par des parties mutuellement méfiantes. Bitcoin dispose bien d'un langage de script rudimentaire (permettant, par exemple, d'exiger plusieurs signatures pour dépenser une transaction — un mécanisme dit *multisig*), mais ce langage est volontairement limité et non Turing-complet (pas de boucles, notamment), ce qui empêche l'exécution de programmes arbitrairement complexes. C'est précisément cette limitation que le concepteur d'Ethereum, Vitalik Buterin, a proposé de dépasser en 2013-2014, en concevant une blockchain dont le langage de script serait Turing-complet.

Déterminisme : une contrainte de conception fondamentale

Une propriété essentielle, souvent sous-estimée par les étudiants découvrant le sujet, est que le code d'un smart contract doit être **strictement déterministe** : exécuté par des milliers de nœuds indépendants dans le monde, à des instants différents, sur des machines différentes, il doit systématiquement produire exactement le même résultat, faute de quoi les nœuds ne pourraient jamais s'accorder sur l'état résultant (rompant la propriété de consensus étudiée au chapitre 3). Cette contrainte a des conséquences concrètes sur ce qu'un smart contract peut ou ne peut pas faire nativement : pas d'accès direct à une horloge « réelle » précise, pas de génération de nombres véritablement aléatoires (une source de vulnérabilité classique, abordée au chapitre 5), pas d'appel réseau vers une ressource externe (d'où le problème de l'oracle, développé en section 6 de ce chapitre).

2. Ethereum : une blockchain programmable

Ethereum (2015) a étendu le modèle de Bitcoin en introduisant une **machine virtuelle Turing-complète** (l'EVM — Ethereum Virtual Machine), permettant d'exécuter des programmes arbitrairement complexes sur la blockchain, et non plus seulement des transactions de transfert de valeur.

Notion de « gas »

Chaque opération exécutée par l'EVM consomme du « gas », payé par l'émetteur de la transaction dans la cryptomonnaie native (Ether). Ce mécanisme a un double rôle : rémunérer les validateurs pour les ressources de calcul consommées, et **empêcher les boucles infinies ou les attaques par déni de service** (un programme épuise son budget de gas avant de pouvoir consommer des ressources indéfiniment — solution pratique au problème de l'arrêt appliqué à un contexte adversarial).

Le problème de l'arrêt, revisité

Le lien avec le problème de l'arrêt (informatique théorique) mérite d'être explicité : il est démontré qu'il n'existe, en général, aucun algorithme capable de déterminer, pour un programme et une entrée arbitraires, si son exécution se terminera ou bouclera indéfiniment. Une blockchain à smart contracts Turing-complets fait face à une version pratique et adversariale de ce problème : un validateur ne peut pas, avant exécution, garantir qu'un contrat soumis par un utilisateur quelconque terminera son exécution en un temps raisonnable — et un attaquant pourrait délibérément soumettre un programme conçu pour boucler indéfiniment, dans le but de paralyser le réseau (attaque par déni de service). Le mécanisme du gas contourne élégamment ce problème indécidable en le rendant **non pertinent** : peu importe qu'un programme termine ou non en théorie, il est de toute façon interrompu de force dès que son budget de gas est épuisé, la transaction étant alors annulée (mais le gas déjà consommé n'est pas remboursé, ce qui dissuade les tentatives malveillantes ou les erreurs de programmation coûteuses).

Chaque opcode a un coût précis

Chaque instruction élémentaire de l'EVM (un « opcode », par exemple additionner deux nombres, lire une variable en mémoire, écrire une variable en stockage persistant) se voit attribuer un coût en gas fixé par le protocole, reflétant approximativement le coût réel de la ressource consommée par les validateurs pour l'exécuter. Un point pédagogique important : l'écriture d'une donnée en **stockage persistant** (`storage`, la partie de l'état d'un contrat qui reste enregistrée sur la blockchain entre deux transactions) coûte typiquement beaucoup plus cher en gas que la manipulation d'une variable en **mémoire temporaire** (`memory`, effacée à la fin de l'exécution de la transaction) — une différence de coût qui a des conséquences directes sur la façon dont un développeur Solidity doit structurer son code pour rester économique, et qui explique pourquoi les boucles itérant sur des structures de données stockées on-chain sont un point d'attention de sécurité particulier (chapitre 5, attaques par déni de service via consommation excessive de gas).

3. Langages et environnement de développement

- **Solidity** : langage le plus utilisé pour écrire des smart contracts sur Ethereum, syntaxe proche du JavaScript/C++.
- **Environnements de développement/test** : Remix (IDE en ligne), Hardhat, Foundry, permettant de développer, tester et déployer sur des réseaux de test avant tout déploiement sur le réseau principal.
- **Réseaux de test (testnets)** : répliques du réseau principal utilisant une monnaie sans valeur réelle, indispensables pour tester un contrat avant déploiement (utilisés en TP3/TP4).

Cycle de vie du développement d'un smart contract

À titre pédagogique, le développement d'un smart contract destiné à un déploiement en production suit typiquement les étapes suivantes, chacune ayant un rôle distinct dans la réduction du risque (les risques spécifiques étant détaillés au chapitre 5) :

1. **Écriture du code** (Solidity ou un langage équivalent) et compilation vers le bytecode EVM.
2. **Tests unitaires locaux**, exécutés sur un nœud de développement local simulant l'EVM (rapide, sans coût réel), vérifiant le comportement attendu de chaque fonction, y compris les cas limites et les tentatives volontaires de faire échouer le contrat.
3. **Déploiement sur un ou plusieurs réseaux de test (testnets)**, avec une monnaie sans valeur réelle obtenue via un « robinet » (*faucet*), permettant de vérifier le comportement du contrat dans des conditions proches du réseau principal (latence réelle, coûts de gas réels bien que la monnaie soit fictive).
4. **Audit de sécurité** (interne et/ou par un tiers spécialisé), combinant revue manuelle et outils automatisés (chapitre 5).
5. **Déploiement sur le réseau principal**, généralement précédé d'un déploiement progressif ou d'une limite de fonds autorisés, le temps de gagner en confiance sur le comportement du contrat en conditions réelles.

Cette prudence méthodique s'explique directement par une propriété déjà mentionnée : un smart contract déployé est, par défaut, immuable — contrairement à une application web classique, il n'existe généralement pas de correctif silencieux possible après coup (chapitre 5 revient en détail sur cette contrainte et sur les mécanismes d'évolutivité qui tentent, avec leurs propres risques, d'y répondre).

4. Exemple simplifié de structure d'un smart contract (Solidity)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract CompteSimple {
    mapping(address => uint256) public soldes;

    function deposer() external payable {
        soldes[msg.sender] += msg.value;
    }

    function retirer(uint256 montant) external {
        require(soldes[msg.sender] >= montant, "Solde insuffisant");
        soldes[msg.sender] -= montant;
        payable(msg.sender).transfer(montant);
    }
}
```

Ce court exemple illustre déjà des notions clés : état persistant sur la blockchain (`mapping`), vérifications explicites (`require`), et manipulation de valeur (Ether) — et prépare directement le chapitre suivant sur la sécurité (l'ordre des opérations dans `retirer` a une importance de sécurité directe).

Éléments de vocabulaire Solidity à connaître

Le tableau suivant résume, à titre de référence, quelques éléments syntaxiques et sémantiques essentiels de Solidity utilisés dans ce chapitre et le suivant :

Élément	Rôle
<code>mapping(clé => valeur)</code>	structure de données associative persistante, analogue à un dictionnaire, mais dont chaque emplacement non initialisé retourne une valeur par défaut (<code>0</code> pour un entier) plutôt qu'une erreur
<code>external</code> / <code>public</code> / <code>internal</code> / <code>private</code>	visibilité d'une fonction ou variable — <code>external</code> n'est callable que depuis l'extérieur du contrat, <code>internal/private</code> restreignent l'accès au contrat (et ses héritiers pour <code>internal</code>)
<code>payable</code>	marque une fonction ou une adresse comme capable de recevoir de l'Ether
<code>msg.sender</code>	adresse ayant initié l'appel de la fonction courante (variable globale fournie par l'EVM)
<code>msg.value</code>	montant d'Ether envoyé avec l'appel courant
<code>require(condition, message)</code>	interrompt l'exécution et annule toutes les modifications d'état effectuées si la condition est fausse, en remboursant le gas non consommé
<code>view</code> / <code>pure</code>	marquent une fonction qui ne modifie pas l'état (<code>view</code>) ou qui ne le lit même pas (<code>pure</code>) — utile pour la lisibilité et pour permettre des appels gratuits en lecture seule

Exemple pédagogique complet : un contrat de vote simplifié

Afin d'illustrer un cas d'usage au-delà du simple transfert de fonds, voici un contrat pédagogique simplifié de vote, dont la logique met en jeu davantage de structures de données et d'états :

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract VoteSimple {
    address public organisateur;
    mapping(address => bool) public aVote;
    mapping(string => uint256) public voix;
    bool public voteOuvert;

    constructor() {
        organisateur = msg.sender; // le dépoyeur du contrat devient l'organisateur
        voteOuvert = true;
    }

    modifier seulementOrganisateur() {
        require(msg.sender == organisateur, "Reserve a l'organisateur");
        _;
    }

    function voter(string calldata candidat) external {
        require(voteOuvert, "Le vote est clos");
        require(!aVote[msg.sender], "Vous avez deja vote");
        aVote[msg.sender] = true;
        voix[candidat] += 1;
    }

    function cloturerVote() external seulementOrganisateur {
        voteOuvert = false;
    }
}
```

Cet exemple introduit la notion de **modificateur** (*modifier*), un mécanisme Solidity permettant de factoriser une condition de contrôle d'accès réutilisable sur plusieurs fonctions — directement pertinent pour la classe de vulnérabilité « contrôle d'accès défaillant » étudiée au chapitre 5. Il illustre également, de façon volontairement simplifiée à des fins pédagogiques, les limites d'un vote purement on-chain : l'anonymat de l'électeur n'est pas assuré (chaque vote est publiquement associé à l'adresse de l'électeur), un point directement repris dans la discussion sur le vote électronique au chapitre 6.

5. Cas d'usage des smart contracts

- **Jetons (tokens)** : représentation d'actifs numériques selon des standards (ERC-20 pour des jetons fongibles, ERC-721 pour des jetons non fongibles/NFT).
- **Finance décentralisée (DeFi)** : prêts, échanges décentralisés (DEX), sans intermédiaire financier traditionnel.
- **Gouvernance décentralisée (DAO)** : votes et décisions automatisées selon des règles codées.
- **Traçabilité** : suivi de chaîne logistique, certification de provenance.

Les standards ERC : un aperçu

Les standards ERC (*Ethereum Request for Comments*) sont des interfaces normalisées, adoptées par la communauté des développeurs, que doit respecter un smart contract pour être reconnu et manipulé de façon uniforme par les portefeuilles, plateformes d'échange et autres contrats de l'écosystème — un rôle analogue à celui d'une interface ou d'un protocole normalisé en génie logiciel classique. Deux standards sont particulièrement structurants :

- **ERC-20** (jetons fongibles) définit un ensemble minimal de fonctions qu'un contrat de jeton doit implémenter (`transfer`, `balanceOf`, `approve`, notamment), permettant à n'importe quel jeton respectant ce standard d'être immédiatement compatible avec l'ensemble des portefeuilles et plateformes de l'écosystème, sans intégration spécifique préalable ;
- **ERC-721** (jetons non fongibles, NFT) définit une interface où chaque jeton émis est identifié individuellement par un numéro unique et n'est pas interchangeable avec un autre jeton du même contrat — adapté à la représentation d'actifs uniques (œuvre numérique, titre de propriété, objet de collection).

Un point pédagogique important : le respect d'un standard ERC est une convention logicielle vérifiée par la présence des bonnes signatures de fonctions, non une garantie de sécurité intrinsèque — un contrat peut parfaitement implémenter l'interface ERC-20 tout en contenant, par ailleurs, l'une des vulnérabilités étudiées au chapitre 5.

6. Le problème de l'oracle

Un smart contract, par conception, ne peut accéder qu'aux données déjà présentes sur la blockchain : il ne peut pas consulter directement une donnée du monde extérieur (cours d'une action, résultat sportif, météo). Un **oracle** est un service tiers chargé d'apporter cette donnée externe sur la blockchain. Ce point est une source de risque majeure : la sécurité de tout le smart contract dépend alors de la fiabilité de l'oracle utilisé, qui redevient un point de confiance centralisé, en tension avec l'objectif initial de décentralisation.

Stratégies d'atténuation du risque d'oracle

Plusieurs approches, chacune avec ses propres compromis, tentent de réduire (sans jamais totalement éliminer) le risque associé à un point de confiance centralisé introduit par un oracle :

- **agrégation de plusieurs sources indépendantes** : combiner (par exemple par médiane) les valeurs rapportées par plusieurs fournisseurs de données indépendants, de façon à ce qu'un attaquant doive compromettre simultanément une majorité d'entre eux pour manipuler la donnée finale ;
- **réseaux d'oracles décentralisés** : faire reporter la donnée par un ensemble de nœuds opérés par des entités distinctes, avec un mécanisme de consensus et d'incitation économique dédié (garantie déposée, pénalisée en cas de donnée manifestement erronée), rapprochant conceptuellement le problème de l'oracle du problème général du consensus étudié au chapitre 3 ;
- **délai et fenêtres de temps moyennées** : plutôt que d'utiliser un prix instantané (manipulable ponctuellement, notamment via un prêt flash, chapitre 5), utiliser une moyenne calculée sur une

fenêtre de temps, rendant la manipulation plus coûteuse car devant être soutenue plus longtemps ;

- **oracles optimistes avec période de contestation** : publier une donnée avec un délai pendant lequel tout observateur peut la contester en apportant une preuve contraire, avant qu'elle ne soit considérée comme définitive par les contrats consommateurs.

Aucune de ces approches ne supprime totalement le problème de fond : par construction, un smart contract ne peut jamais avoir une certitude cryptographique équivalente à celle qu'il a sur ses propres données internes lorsqu'il s'agit d'une information provenant du monde extérieur — un rappel utile face au discours parfois trop optimiste sur la capacité de la blockchain à « garantir la vérité ».

À retenir

- Un smart contract est un programme auto-exécutable et strictement déterministe (contrainte nécessaire pour que tous les nœuds du réseau convergent vers le même résultat), pas nécessairement un contrat juridique au sens classique.
- Le mécanisme de gas rémunère l'exécution, avec un coût par opération reflétant approximativement la ressource réellement consommée, et contourne élégamment le problème théorique de l'arrêt en interrompant de force toute exécution dépassant le budget alloué.
- Les standards ERC (ERC-20 pour les jetons fongibles, ERC-721 pour les jetons non fongibles) normalisent les interfaces des smart contracts pour assurer leur interopérabilité avec l'écosystème, sans garantir pour autant leur sécurité intrinsèque.
- Le problème de l'oracle rappelle qu'un smart contract ne résout la confiance que pour les données déjà présentes sur la chaîne, pas pour les données du monde réel qui y sont importées ; plusieurs stratégies (agrégation, décentralisation des oracles, moyennes temporelles) atténuent ce risque sans jamais l'éliminer complètement.
- La prudence méthodique du cycle de développement (tests, testnets, audit avant déploiement) découle directement de l'immutabilité par défaut des smart contracts déployés, un point développé en détail au chapitre 5.

Chapitre 5 — Sécurité et vulnérabilités des smart contracts

1. Pourquoi la sécurité des smart contracts est particulièrement critique

Un smart contract déployé sur une blockchain publique est, par construction, **immuable et son code est public** : contrairement à une application classique, il n'est généralement pas possible de corriger une faille par un simple correctif déployé discrètement — le code vulnérable reste actif et visible de tous jusqu'à une migration complexe. De plus, les contrats manipulent souvent directement des fonds réels, ce qui en fait des cibles à forte valeur immédiate pour un attaquant.

Cette combinaison de facteurs — code public, immutabilité, valeur financière directe, exécution automatique sans intervention humaine possible pour interrompre une transaction en cours — distingue nettement le modèle de menace des smart contracts de celui d'une application web classique étudiée dans les modules précédents de ce cours. Il est utile de comparer explicitement ces deux contextes :

Critère	Application web classique	Smart contract déployé
Visibilité du code source	généralement non public (code compilé côté serveur)	bytecode public par construction ; code source souvent publié volontairement pour la transparence
Correction après découverte d'une faille	correctif déployé rapidement, souvent de façon transparente pour l'utilisateur	migration complexe, souvent impossible sans mécanisme d'évolutivité prévu à l'avance
Réversibilité d'une transaction frauduleuse	souvent possible (annulation, remboursement, intervention d'un administrateur)	généralement impossible une fois la transaction confirmée (chapitre 2)
Fenêtre d'exploitation	limitée par les capacités de détection et réaction de l'équipe opérationnelle	potentiellement illimitée tant que le contrat reste actif et non corrigé

Cette comparaison justifie pourquoi la démarche d'audit **avant** déploiement (section 7 de ce chapitre) occupe une place centrale dans la sécurité des smart contracts, alors qu'une application web classique peut davantage s'appuyer sur une détection et une réaction rapides après incident (modules *Détection d'intrusion* et *Réponse aux incidents*).

Mécanismes d'évolutivité et leurs propres risques

Face à la rigidité de l'immutabilité, plusieurs patrons de conception permettent de prévoir, dès le déploiement initial, une possibilité de mise à jour ultérieure du code — au prix d'une complexité et d'une surface d'attaque supplémentaires :

- **contrats proxy (délégation)** : un contrat « proxy », dont l'adresse est celle communiquée aux utilisateurs, délègue l'exécution de sa logique à un second contrat d'implémentation dont

l'adresse peut être changée par un administrateur autorisé ; cela permet de faire évoluer la logique métier sans changer l'adresse publique du contrat, mais concentre un pouvoir important (et donc un risque) entre les mains de qui contrôle le changement d'implémentation ;

- **pause d'urgence (circuit breaker)** : une fonction, protégée par un contrôle d'accès strict, permettant de suspendre temporairement certaines opérations sensibles du contrat en cas de détection d'une attaque en cours, limitant les pertes le temps d'une analyse approfondie ;
- **limites de montant et listes blanches** : contraindre dès la conception les montants manipulables ou les adresses autorisées à interagir avec les fonctions les plus sensibles, réduisant l'impact maximal d'une faille non encore découverte.

Chacun de ces mécanismes introduit lui-même une nouvelle surface d'attaque potentielle (par exemple, un contrôle d'accès défaillant sur la fonction de changement d'implémentation d'un proxy annule tout l'intérêt du mécanisme) — un rappel que la sécurité d'un système complexe se raisonne toujours de façon globale, jamais mécanisme par mécanisme isolément.

2. Réentrance (reentrancy)

Vulnérabilité historique majeure (à l'origine du piratage « The DAO » en 2016, environ 60 millions de dollars de l'époque). Principe : un contrat appelle un contrat externe (ex. pour transférer des fonds) **avant** d'avoir mis à jour son propre état interne ; le contrat externe malveillant peut alors rappeler la fonction d'origine de façon récursive, exploitant le fait que l'état (le solde) n'a pas encore été mis à jour.

```
// Version vulnérable
function retirer(uint256 montant) external {
    require(soldes[msg.sender] >= montant, "Solde insuffisant");
    payable(msg.sender).transfer(montant); // appel externe AVANT la mise à jour de l'é
    soldes[msg.sender] -= montant;          // trop tard : un contrat malveillant a pu
}
```

Contre-mesure : appliquer le patron **checks-effects-interactions** (vérifications, puis mise à jour de l'état interne, puis seulement interaction externe) — c'est exactement l'ordre correct déjà utilisé dans l'exemple du chapitre 4.

```
// Version corrigée, appliquant le patron checks-effects-interactions
function retirer(uint256 montant) external {
    require(soldes[msg.sender] >= montant, "Solde insuffisant"); // 1. vérifications (c
    soldes[msg.sender] -= montant;                                // 2. mise à jour de
    payable(msg.sender).transfer(montant);                        // 3. interaction ext
}
```

Pourquoi `transfer` ne suffit pas toujours : le rôle du verrou de réentrance

Historiquement, une contre-mesure complémentaire consistait à utiliser la fonction `transfer` (plutôt que `call`) pour envoyer des fonds, car elle limite le gas transmis à l'appel externe (2300 unités à l'époque), rendant impossible l'exécution d'une logique complexe — dont un rappel récursif — dans le contrat destinataire. Cette contre-mesure est aujourd'hui considérée comme fragile et insuffisante à elle seule (le coût de certaines opérations EVM a évolué, et de nombreux contrats destinataires légitimes,

comme les portefeuilles multi-signatures, ont eux-mêmes besoin de plus de gas que cette limite pour fonctionner correctement) ; le patron checks-effects-interactions reste la contre-mesure de référence, indépendante des variations de coût du gas. Une seconde contre-mesure complémentaire, souvent utilisée en défense en profondeur, consiste à ajouter un **verrou de réentrance** (*reentrancy guard*) explicite :

```
contract CompteProtege {
    mapping(address => uint256) public soldes;
    bool private enCoursDExecution; // verrou de réentrance

    modifier nonReentrant() {
        require(!enCoursDExecution, "Appel reentrant detecte");
        enCoursDExecution = true;
        _;
        enCoursDExecution = false;
    }

    function retirer(uint256 montant) external nonReentrant {
        require(soldes[msg.sender] >= montant, "Solde insuffisant");
        soldes[msg.sender] -= montant;
        payable(msg.sender).transfer(montant);
    }
}
```

Ce modificateur `nonReentrant` empêche explicitement qu'un même appel « en cours » soit appelé récursivement, quelle que soit la fonction externe utilisée pour transférer les fonds — une défense indépendante et complémentaire au patron checks-effects-interactions, la combinaison des deux étant considérée comme une bonne pratique robuste.

Réentrance inter-fonctions et réentrance en lecture seule

Un piège pédagogique fréquent consiste à croire qu'un verrou de réentrance appliqué à une seule fonction protège l'ensemble du contrat. En réalité, si deux fonctions distinctes partagent un état commun et qu'une seule des deux est protégée par le patron checks-effects-interactions ou par un verrou de réentrance, un attaquant peut parfois exploiter la fonction non protégée pendant l'exécution suspendue de la première — une variante dite **réentrance inter-fonctions**. Une variante plus subtile encore, la **réentrance en lecture seule** (*read-only reentrancy*), exploite le fait qu'une fonction externe appelée pendant un appel en cours peut lire un état temporairement incohérent (par exemple, un solde déjà débité mais un transfert pas encore confirmé) via une fonction de lecture (`view`) d'un tout autre contrat qui se fie à cet état pour son propre calcul — illustrant que le raisonnement de sécurité doit porter sur l'ensemble des contrats en interaction, pas uniquement sur celui contenant la fonction vulnérable la plus évidente.

3. Débordement/souppassement d'entier (overflow/underflow)

Historiquement, un compteur non signé en Solidity pouvait « déborder » silencieusement (revenir à zéro après le maximum, ou à une valeur maximale après une soustraction sous zéro), exploité pour créer des soldes ou des quantités de jetons artificiellement énormes. Depuis Solidity 0.8, ces

vérifications sont effectuées automatiquement par défaut (le compilateur annule la transaction en cas de dépassement), mais le risque reste réel pour du code utilisant des versions antérieures ou des blocs `unchecked` mal justifiés.

```
// Exemple pédagogique illustrant le risque historique (avant Solidity 0.8, ou dans un
uint8 compteur = 255;          // uint8 : valeur maximale représentable = 255
unchecked {
    compteur += 1;              // débordement silencieux : compteur devient 0, sans erreur
}

// Depuis Solidity >= 0.8, sans le mot-clé `unchecked`, la même opération
// provoquerait automatiquement l'annulation de la transaction (revert).
```

Le mot-clé `unchecked` reste disponible depuis Solidity 0.8 pour des raisons d'optimisation du coût en gas dans des cas où le développeur peut prouver, par la logique du programme, qu'un dépassement est impossible (par exemple un compteur de boucle borné par une valeur elle-même petite) ; son usage doit néanmoins être systématiquement justifié et documenté lors d'un audit, car il réintroduit délibérément le risque historique dans le seul segment de code qu'il englobe.

4. Contrôle d'accès défaillant

Une fonction sensible (ex. modification du propriétaire du contrat, retrait de fonds réservés à l'administrateur) sans vérification appropriée des droits de l'appelant (`require(msg.sender == propriétaire)`) permet à n'importe quel utilisateur de l'exécuter. C'est l'équivalent, dans le monde des smart contracts, du contrôle d'accès défaillant classé numéro un de l'OWASP Top 10 (module *Audit organisation et technique*).

```

// Version vulnérable : n'importe qui peut devenir propriétaire du contrat
contract GestionVulnerable {
    address public proprietaire;

    constructor() {
        proprietaire = msg.sender;
    }

    function changerProprietaire(address nouveauProprietaire) external {
        proprietaire = nouveauProprietaire; // aucune vérification de l'appelant !
    }
}

// Version corrigée
contract GestionSecurisee {
    address public proprietaire;

    constructor() {
        proprietaire = msg.sender;
    }

    modifier seulementProprietaire() {
        require(msg.sender == proprietaire, "Reserve au proprietaire");
        _;
    }

    function changerProprietaire(address nouveauProprietaire) external seulementProprietaire {
        require(nouveauProprietaire != address(0), "Adresse invalide");
        proprietaire = nouveauProprietaire;
    }
}

```

Cet exemple illustre également une bonne pratique complémentaire : vérifier que l'adresse fournie n'est pas l'adresse nulle (`address(0)`), une erreur de saisie ou de logique qui, dans un contrat mal protégé, peut rendre définitivement inaccessibles des fonctions administratives (aucune clé privée ne correspond à l'adresse nulle).

Fonctions non initialisées et visibilité par défaut

Une variante historique de cette classe de vulnérabilité, moins fréquente avec les versions récentes du compilateur qui imposent une visibilité explicite, mérite d'être connue à titre pédagogique : dans d'anciennes versions de Solidity, une fonction sans mot-clé de visibilité explicite était `public` par défaut, ce qui pouvait rendre accessible depuis l'extérieur une fonction destinée à un usage strictement interne (par exemple une fonction d'initialisation censée n'être appelée qu'une seule fois par le déploiement lui-même). Ce type d'erreur illustre un principe de sécurité plus général, déjà rencontré dans d'autres modules de ce cours : préférer systématiquement une **politique de refus par défaut** (*deny by default*, restreindre explicitement l'accès et n'ouvrir que ce qui est nécessaire) à une politique d'autorisation par défaut.

5. Attaques par manipulation d'oracle de prix

Un contrat DeFi qui détermine un prix d'actif à partir d'une source manipulable (ex. la liquidité instantanée d'un seul échange décentralisé) peut être trompé par un attaquant qui manipule temporairement ce prix (souvent via un **prêt flash**, un emprunt sans garantie remboursé dans la même transaction) pour déclencher des conditions favorables artificielles dans le contrat cible.

Anatomie d'un scénario pédagogique de manipulation par prêt flash

Afin de rendre ce mécanisme concret, considérons un scénario entièrement fictif, à but pédagogique uniquement (les montants et le nom du protocole sont inventés) :

1. Un protocole fictif « PrêtDeFi » permet d'emprunter un actif en garantissant un autre actif, en évaluant la valeur de la garantie à partir du prix instantané observé sur un unique échange décentralisé de faible liquidité.
2. Un attaquant contracte un prêt flash important dans un autre actif, sans garantie, remboursable obligatoirement avant la fin de la même transaction.
3. Il utilise ces fonds empruntés pour acheter massivement, en une seule opération, l'actif utilisé comme garantie sur ce petit échange décentralisé, en faisant artificiellement grimper son prix instantané sur cet échange précis.
4. Toujours dans la même transaction, il dépose ce même actif (dont le prix affiché est maintenant artificiellement élevé) en garantie sur le protocole « PrêtDeFi », et emprunte un montant excessif, disproportionné par rapport à la valeur réelle de sa garantie.
5. Il rembourse le prêt flash initial (obligatoire dans la même transaction), et conserve la différence empruntée en excès — une perte nette pour les autres utilisateurs du protocole « PrêtDeFi ».

Ce scénario, bien que fictif dans ses détails numériques, correspond à une classe d'attaques réellement documentée dans l'écosystème DeFi et illustre pourquoi la conception d'un oracle de prix robuste (agrégation de plusieurs sources, moyennes temporelles, cf. chapitre 4) est une décision de sécurité de premier ordre, et non un détail d'implémentation secondaire, pour tout contrat manipulant des montants financiers déterminés par un prix externe.

6. Attaques par déni de service et boucles coûteuses en gas

Une fonction itérant sur une structure de données dont la taille peut être manipulée par un attaquant (ex. une liste d'utilisateurs qu'il peut faire croître) peut consommer plus de gas que le maximum autorisé par bloc, rendant la fonction définitivement inexécutable — un déni de service permanent et non corrigible sans migration du contrat.

```

// Version vulnérable : la boucle grandit avec le nombre d'inscrits, sans limite
contract DistributionVulnerable {
    address[] public inscrits;

    function sInscrire() external {
        inscrits.push(msg.sender);
    }

    function distribuerRecompense() external {
        for (uint256 i = 0; i < inscrits.length; i++) {
            payable(inscrits[i]).transfer(1 ether); // coûte de plus en plus de gas à m
        }
        // Au-delà d'un certain nombre d'inscrits, cette boucle dépasse la limite de ga
        // autorisée par bloc et la fonction devient définitivement inexécutable.
    }
}

// Version corrigée : modèle "pull" plutôt que "push" – chaque utilisateur retire lui-m
contract DistributionCorrigee {
    mapping(address => bool) public inscrits;
    mapping(address => uint256) public recompensesDisponibles;

    function sInscrire() external {
        inscrits[msg.sender] = true;
        recompensesDisponibles[msg.sender] = 1 ether;
    }

    function retirerRecompense() external {
        uint256 montant = recompensesDisponibles[msg.sender];
        require(montant > 0, "Aucune recompense disponible");
        recompensesDisponibles[msg.sender] = 0; // état mis à jour avant l'interactio
        payable(msg.sender).transfer(montant);
    }
}

```

Ce couple d'exemples illustre un principe de conception plus général, connu sous le nom de **patron pull-over-push** : plutôt que pour un contrat de pousser proactivement des fonds vers une liste potentiellement non bornée de destinataires (coût cumulé porté par une seule transaction, donc un seul appelant), il est préférable de laisser chaque bénéficiaire retirer lui-même sa part dans sa propre transaction (coût réparti, chaque appelant ne payant que pour sa propre opération). Ce principe réapparaît fréquemment dans la conception de contrats DeFi robustes.

7. Démarche d'audit d'un smart contract

1. Revue manuelle du code, en confrontant systématiquement chaque fonction sensible aux classes de vulnérabilités ci-dessus.
2. Analyse statique automatisée (outils dédiés type Slither, MythX).
3. Tests unitaires et de fuzzing (génération automatique d'entrées pour détecter des comportements inattendus).
4. Vérification formelle pour les contrats à très haute valeur (preuve mathématique de propriétés de sécurité).

5. Programme de récompense pour la découverte de vulnérabilités (*bug bounty*) avant et après déploiement.

Les limites propres à chaque outil d'audit

Il est pédagogiquement important de ne présenter aucun de ces outils comme une garantie absolue, mais bien comme des instruments complémentaires, chacun avec ses angles morts caractéristiques :

- l'**analyse statique automatisée** détecte efficacement les classes de vulnérabilités déjà connues et « signées » dans ses règles (réentrance simple, visibilité incorrecte, usages dangereux de primitives connues), mais peut manquer des failles logiques propres au métier du contrat (par exemple une condition d'éligibilité mal pensée sur le plan économique) et génère parfois de faux positifs à trier manuellement ;
- le **fuzzing** (génération automatique d'entrées, y compris de séquences d'appels de fonctions) est efficace pour découvrir des combinaisons d'entrées non anticipées par les développeurs, mais son efficacité dépend directement de la qualité des invariants définis par l'auditeur (les propriétés que le fuzzer doit essayer de violer) ;
- la **vérification formelle** offre la garantie la plus forte (une preuve mathématique qu'une propriété précisément spécifiée est respectée dans tous les cas), mais est coûteuse en temps d'ingénierie, ne couvre que les propriétés explicitement formalisées (une propriété oubliée dans la spécification reste un angle mort), et reste réservée en pratique aux contrats à très haute valeur ou à très fort risque systémique ;
- la **revue manuelle** reste irremplaçable pour évaluer la cohérence globale de la logique métier et repérer des vulnérabilités inédites, mais dépend fortement de l'expertise et de la disponibilité en temps de l'auditeur, et ne passe pas à l'échelle sur de très gros codebases sans être combinée aux approches automatisées.

Un audit rigoureux combine typiquement plusieurs de ces approches plutôt que de s'appuyer sur une seule, exactement comme un test de pénétration classique (module *Sécurité offensive*) combine différentes techniques complémentaires plutôt qu'un unique outil automatisé.

Un rappel historique : l'attaque de The DAO

L'attaque de The DAO (2016), déjà mentionnée en introduction de ce chapitre comme illustration de la vulnérabilité de réentrance, mérite un rappel de son importance historique pour le domaine : au-delà de la perte financière directe, cet incident a provoqué un débat de gouvernance majeur au sein de la communauté Ethereum sur la conduite à tenir face à une exploitation manifeste d'une faille de code — fallait-il considérer que « le code fait loi » (*code is law*) et laisser les fonds détournés à l'attaquant, puisque son action respectait strictement les règles du code déployé, ou bien intervenir pour annuler les conséquences de l'attaque au nom de l'intention originelle des utilisateurs lésés ? Ce débat a conduit à un hard fork du réseau (chapitre 3) et à la coexistence, depuis lors, de deux chaînes distinctes issues de ce désaccord. Cet épisode reste, encore aujourd'hui, une référence pédagogique incontournable pour illustrer à la fois la criticité technique de la réentrance et la dimension de gouvernance propre aux blockchains publiques, qui dépasse la seule question technique de la sécurité du code.

À retenir

- La réentrance, le contrôle d'accès défaillant, la manipulation d'oracle de prix et les attaques par déni de service via consommation excessive de gas sont des classes de vulnérabilités récurrentes, à l'origine de pertes financières réelles considérables dans l'écosystème des smart contracts.
- L'immutabilité des smart contracts déployés rend la prévention (audit avant déploiement) bien plus critique que pour une application web classique corrigible a posteriori ; des mécanismes d'évolutivité (proxy, pause d'urgence) existent mais introduisent leur propre surface d'attaque.
- Le patron checks-effects-interactions, éventuellement complété par un verrou de réentrance explicite, est la contre-mesure de référence contre la réentrance (y compris ses variantes inter-fonctions et en lecture seule), illustrant l'importance de l'ordre des opérations dans le code.
- Le patron pull-over-push (laisser chaque bénéficiaire retirer ses fonds plutôt que de les distribuer en boucle) prévient les attaques par déni de service liées à une consommation de gas non bornée.
- Aucun outil d'audit (analyse statique, fuzzing, vérification formelle, revue manuelle) n'est suffisant isolément : une démarche d'audit rigoureuse combine plusieurs approches complémentaires, exactement comme un test de pénétration classique.

Chapitre 6 — Applications, enjeux et limites

1. Panorama des applications au-delà des cryptomonnaies

- **Finance décentralisée (DeFi)** : prêts, échanges, produits dérivés sans intermédiaire financier centralisé — avec les risques de sécurité vus au chapitre 5 et une volatilité/régulation encore incertaine.
- **Traçabilité de chaîne logistique** : suivi de provenance de produits (agroalimentaire, textile, minerais), où la blockchain garantit l'intégrité des enregistrements une fois saisis, sans garantir la véracité de la saisie initiale (problème de l'oracle, rappel chapitre 4).
- **Identité numérique décentralisée (self-sovereign identity)** : permettre à un individu de contrôler et prouver sélectivement des attributs de son identité sans dépendre d'un registre central unique.
- **Certification et notariation** : preuve d'existence et d'horodatage d'un document à une date donnée (ancrage de l'empreinte du document sur une blockchain publique) — usage simple et robuste, sans manipuler de valeur financière.
- **Vote électronique** : cas d'usage souvent évoqué mais particulièrement délicat en pratique, car il combine des exigences difficilement conciliables (anonymat de l'électeur, vérifiabilité publique du résultat, résistance à la coercition) — objet de recherche active plutôt que de déploiements matures à grande échelle.

Le contrat de vote du chapitre 4, revisité à la lumière de ces exigences

Le contrat pédagogique `VoteSimple` présenté au chapitre 4 illustre concrètement les tensions évoquées ci-dessus : il assure une vérifiabilité publique parfaite (chacun peut recompter les voix directement depuis la blockchain), mais aucun anonymat (chaque vote reste associé de façon permanente à l'adresse de l'électeur qui l'a émis) et aucune résistance à la coercition (un tiers pourrait exiger d'un électeur qu'il prouve, en lui montrant la transaction correspondante, avoir voté dans un sens donné). Concilier ces trois exigences simultanément nécessite des constructions cryptographiques bien plus sophistiquées (preuves à divulgation nulle de connaissance, signatures en anneau, notamment), qui dépassent le cadre de ce cours introductif mais expliquent pourquoi le vote électronique décentralisé reste, à ce jour, davantage un sujet de recherche qu'une solution mature déployée à grande échelle pour des élections officielles.

Identité numérique décentralisée : approfondissement

Le concept de *self-sovereign identity* mérite d'être détaillé davantage, car il illustre un usage de la blockchain qui ne repose pas nécessairement sur la manipulation de valeur financière. L'idée centrale est de permettre à un individu de contrôler directement des **attestations vérifiables** (par exemple, un diplôme délivré par une université, ou une preuve de majorité) émises et signées cryptographiquement par une autorité de confiance (l'université, l'État), sans que cette autorité doive être interrogée à chaque vérification ultérieure. La blockchain intervient alors typiquement non pas pour stocker directement les données personnelles (ce qui poserait un problème majeur au regard du droit à l'effacement des

données personnelles, développé en section 3), mais pour ancrer un registre public des clés de signature des autorités habilitées à émettre de telles attestations, ou pour révoquer une attestation compromise — un usage plus restreint et mieux délimité que ne le laisse parfois entendre la communication marketing autour de ce sujet.

2. Trilemme de la blockchain (scalabilité, sécurité, décentralisation)

Les concepteurs de blockchains publiques font face à un compromis reconnu : il est difficile d'optimiser simultanément la **scalabilité** (nombre de transactions traitées par seconde), la **sécurité** et la **décentralisation**. Les architectures existantes privilégient généralement deux de ces trois propriétés au détriment de la troisième (ex. Bitcoin privilégie sécurité et décentralisation au prix d'une faible scalabilité ; certaines blockchains à permission privilégient scalabilité et sécurité au prix d'une décentralisation réduite).

Pourquoi ce compromis est-il structurel et non simplement un problème d'ingénierie non résolu ?

Il est pédagogiquement utile d'expliquer *pourquoi* ce trilemme n'est pas qu'une limite technologique provisoire, mais découle de contraintes structurelles reliées aux chapitres précédents de ce cours :

- augmenter le **débit** de transactions traitées implique généralement d'augmenter la taille des blocs ou de réduire l'intervalle de temps entre deux blocs ;
- augmenter la taille des blocs ou réduire l'intervalle entre blocs augmente le temps nécessaire à leur **propagation** sur l'ensemble du réseau pair-à-pair, augmentant mécaniquement la probabilité de forks accidentels (chapitre 3) ;
- pour limiter ce risque, il faudrait réduire le nombre de nœuds participant à la validation (moins de nœuds à qui propager, moins de latence de convergence) — ce qui revient précisément à sacrifier la **décentralisation** ;
- à l'inverse, maintenir un grand nombre de nœuds validateurs indépendants et géographiquement dispersés (forte décentralisation) impose de conserver des blocs suffisamment petits et un intervalle de temps suffisamment grand pour que la propagation reste fiable, ce qui plafonne mécaniquement le débit maximal.

Ce raisonnement, volontairement simplifié, illustre que le trilemme n'oppose pas des choix arbitraires mais des contraintes physiques et réseau bien réelles (latence de propagation, bande passante disponible chez chaque nœud) — un rappel utile face aux discours commerciaux annonçant parfois avoir « résolu » le trilemme sans compromis, alors qu'il s'agit presque toujours d'un déplacement du curseur vers un point différent du compromis, pas d'une élimination du compromis lui-même.

Approches de mise à l'échelle (aperçu)

- **Solutions de couche 2 (Layer 2)** : traiter la majorité des transactions hors chaîne principale, en n'ancrant sur la chaîne principale qu'un résumé périodique (ex. rollups sur Ethereum).
- **Sharding** : répartir l'état et le traitement entre plusieurs sous-réseaux (« shards ») traités en parallèle.

Approfondissement : le principe des rollups

Les *rollups*, mentionnés ci-dessus comme exemple de solution de couche 2, méritent d'être détaillés à titre d'illustration pédagogique du principe général des solutions hors chaîne. Le principe consiste à exécuter et regrouper (« rouler ») un grand nombre de transactions hors de la chaîne principale, sur un réseau secondaire dédié, puis à ne publier sur la chaîne principale qu'un résumé compact de l'effet cumulé de ces transactions (par exemple une nouvelle racine d'état, au sens du state trie présenté au chapitre 2), accompagné d'une preuve de la validité de ce résumé. Deux grandes familles existent, se distinguant par la nature de cette preuve :

- les **rollups optimistes** publient le résumé en supposant qu'il est correct par défaut, mais ouvrent une fenêtre de contestation pendant laquelle tout observateur peut soumettre une preuve de fraude s'il détecte une exécution incorrecte, auquel cas le résumé contesté est annulé ;
- les **rollups à validité (zk-rollups)** accompagnent chaque résumé d'une preuve cryptographique à divulgation nulle de connaissance, vérifiable rapidement par la chaîne principale, garantissant mathématiquement la correction du résumé sans attendre de fenêtre de contestation ni faire confiance à un tiers.

Dans les deux cas, la chaîne principale continue de jouer son rôle de source ultime de sécurité et de disponibilité des données (elle reste seule garante de l'ordre final et de l'intégrité globale), tandis que l'essentiel du volume de calcul et de transactions est déporté hors chaîne — un exemple concret de la façon dont l'écosystème contourne partiellement le trilemme en séparant les rôles entre plusieurs couches de réseau plutôt qu'en cherchant une solution monolithique unique.

Approfondissement : limites pratiques du sharding

Le sharding, bien qu'il permette en théorie un débit proportionnel au nombre de sous-réseaux (« shards »), introduit des difficultés d'ingénierie substantielles qui expliquent sa mise en œuvre progressive et prudente dans l'écosystème : une transaction impliquant des comptes répartis sur deux shards différents (une **transaction inter-shard**) nécessite une coordination supplémentaire, plus coûteuse et plus lente qu'une transaction interne à un seul shard ; la sécurité de chaque shard pris individuellement doit rester suffisante (un attaquant ne doit pas pouvoir concentrer une majorité de puissance de validation sur un seul shard plus vulnérable, un risque parfois appelé attaque « à un seul shard ») ; enfin, la disponibilité et la cohérence des données doivent être garanties à travers l'ensemble des shards pour que le réseau conserve une vue globale fiable de son propre état.

3. Enjeux réglementaires

- **Lutte contre le blanchiment (AML) et connaissance du client (KYC)** : tension entre pseudonymat des blockchains publiques et obligations réglementaires imposées aux plateformes d'échange.
- **Qualification juridique des cryptoactifs** : selon les juridictions, un jeton peut être qualifié différemment (monnaie, valeur mobilière, actif numérique sui generis), avec des conséquences fiscales et réglementaires très différentes.
- **Protection des consommateurs** : volatilité, risque de perte totale et irréversible en cas d'erreur de manipulation (perte de clé privée) ou d'escroquerie (absence de recours équivalent à un

système bancaire traditionnel).

- **Encadrement progressif** : plusieurs juridictions ont mis en place ou développent des cadres réglementaires dédiés aux actifs numériques et aux prestataires de services associés ; la situation évolue rapidement et diffère fortement d'un pays à l'autre.

La tension pseudonymat/traçabilité en détail

Un point mérite d'être approfondi pour dissiper une confusion fréquente : une blockchain publique comme Bitcoin ou Ethereum n'est en réalité **pas anonyme, mais pseudonyme**. Chaque adresse est un identifiant durable (une chaîne de caractères dérivée d'une clé publique, chapitre 2), et l'intégralité des transactions associées à cette adresse est publiquement consultable et immuable. Ce pseudonymat offre une confidentialité bien moindre qu'il n'y paraît de prime abord : des techniques d'analyse de graphe des transactions (regrouper des adresses probablement contrôlées par un même acteur, à partir de motifs de dépense caractéristiques) permettent, dans de nombreux cas documentés, de relier une adresse à une identité réelle, en particulier au moment où des fonds transitent par une plateforme d'échange soumise à des obligations de connaissance du client (KYC). C'est précisément cette combinaison — pseudonymat des transactions sur la chaîne, mais identification obligatoire aux points d'entrée et de sortie que constituent les plateformes réglementées — qui structure l'essentiel de l'approche réglementaire actuelle de la lutte contre le blanchiment appliquée aux cryptoactifs, plutôt qu'une tentative d'interdire ou de casser techniquement le pseudonymat lui-même.

4. Limites techniques et environnementales

- Consommation énergétique significative pour les réseaux en preuve de travail (chapitre 3), bien que la transition vers la preuve d'enjeu réduise fortement cet impact pour les réseaux concernés.
- Irréversibilité des transactions : un avantage en termes de finalité, mais un risque important en cas d'erreur humaine ou de vol (pas de « bouton d'annulation »).
- Complexité d'usage pour l'utilisateur final (gestion de clés privées, absence de recours en cas de perte), un frein réel à l'adoption grand public.

Le stockage : une ressource qui ne cesse de croître

Une limite technique moins souvent discutée que le débit ou l'énergie mérite d'être signalée : dans une blockchain publique classique (sans mécanisme d'élagage spécifique), chaque nœud complet doit conserver l'intégralité de l'historique depuis le tout premier bloc pour pouvoir valider indépendamment n'importe quelle transaction — un volume de données qui croît de façon monotone au fil du temps, sans jamais diminuer. Cette croissance continue pose un dilemme structurel : plus l'historique devient volumineux, plus il devient coûteux (en stockage, en bande passante) pour un nouveau participant de rejoindre le réseau en tant que nœud complet, ce qui tend à concentrer, avec le temps, le rôle de validation complète chez un nombre plus restreint d'acteurs disposant des ressources nécessaires — une tension directe avec l'objectif de décentralisation à long terme, distincte du trilemme scalabilité/sécurité/décentralisation présenté en section 2 mais qui lui est étroitement liée.

Sécurité de la clé privée : le talon d'Achille de l'utilisateur final

La complexité d'usage mentionnée ci-dessus mérite d'être reliée explicitement au module *Cryptographie* de ce cours : la sécurité de l'intégralité des fonds associés à une adresse repose entièrement sur la confidentialité d'une seule clé privée (ou de la phrase mnémonique qui permet de la régénérer). Contrairement à un compte bancaire classique, il n'existe généralement aucun mécanisme de récupération de mot de passe oublié, aucune procédure de contestation d'une opération frauduleuse déjà confirmée, et aucune assurance obligatoire couvrant la perte. Cette situation déplace une responsabilité de sécurité normalement assumée par une institution financière (protection de l'infrastructure, procédures de recours) directement sur l'utilisateur individuel, qui doit lui-même appliquer de bonnes pratiques de gestion de clés (stockage hors ligne dans un « portefeuille froid » pour des montants significatifs, sauvegardes multiples de la phrase mnémonique, méfiance envers le hameçonnage ciblant spécifiquement les détenteurs de cryptoactifs) — un exemple concret des principes de gestion de clés étudiés de façon plus générale dans le module *Cryptographie*, appliqué ici à un contexte où l'erreur est immédiatement et irréversiblement sanctionnée financièrement.

5. Regard critique : quand la blockchain est-elle pertinente ?

Une grille de questions utile avant d'adopter une solution blockchain pour un projet donné : a-t-on réellement besoin de décentralisation, ou une base de données classique avec un tiers de confiance suffit-elle ? Plusieurs parties mutuellement méfiantes doivent-elles partager un registre sans intermédiaire ? L'immutabilité est-elle un avantage réel pour ce cas d'usage, ou une contrainte gênante (ex. droit à l'effacement en matière de données personnelles) ? De nombreux projets « blockchain » annoncés ces dernières années auraient pu être résolus plus simplement par une architecture centralisée classique.

Une grille de décision synthétique

Le tableau suivant formalise, à titre d'outil pédagogique pour ce cours, la grille de questions ci-dessus sous une forme structurée, utilisable comme point de départ pour l'analyse d'un projet fictif proposé en exercice :

Question	Si la réponse est « non »	Si la réponse est « oui »
Plusieurs parties mutuellement méfiantes doivent-elles partager un état commun sans intermédiaire de confiance unique ?	une base de données classique, éventuellement répliquée, répond probablement mieux au besoin	la décentralisation apporte une valeur réelle à considérer
Le débit de transactions requis est-il compatible avec les mécanismes de consensus disponibles (chapitre 3), éventuellement via une solution de couche 2 ?	risque de goulot d'étranglement majeur, à anticiper dès la conception	la contrainte de scalabilité est gérable
L'immutabilité est-elle un avantage recherché (auditabilité, non-répudiation) plutôt qu'une contrainte (droit à l'effacement, correction d'erreurs) ?	l'immutabilité pourrait devenir un passif juridique ou opérationnel	l'immutabilité renforce la proposition de valeur du projet

Question	Si la réponse est « non »	Si la réponse est « oui »
Les données sensibles peuvent-elles rester hors chaîne (hachées ou référencées uniquement) plutôt que stockées en clair sur un registre public ?	risque de conflit avec la réglementation sur les données personnelles	la conception peut concilier transparence et protection des données

Cette grille ne prétend pas fournir une réponse automatique, mais structurer la discussion critique attendue de tout futur ingénieur confronté à une proposition de projet « blockchain » — un exercice d'esprit critique directement transposable aux travaux pratiques de ce module.

À retenir

- Les applications de la blockchain dépassent les cryptomonnaies (identité décentralisée, traçabilité, certification) mais restent souvent confrontées au problème de l'oracle (chapitre 4) et au trilemme scalabilité/sécurité/décentralisation, un compromis structurel lié aux contraintes physiques de propagation réseau, pas une simple limite d'ingénierie provisoire.
- Les solutions de couche 2 (rollups optimistes ou à validité) et le sharding atténuent partiellement ce trilemme en répartissant les rôles entre plusieurs couches ou sous-réseaux, sans jamais l'éliminer complètement.
- Le pseudonymat (et non l'anonymat) des blockchains publiques structure l'approche réglementaire actuelle de lutte contre le blanchiment, centrée sur l'identification aux points d'entrée/sortie (plateformes d'échange) plutôt que sur la chaîne elle-même.
- Le cadre réglementaire des cryptoactifs est encore en construction et varie fortement selon les juridictions ; la qualification juridique d'un jeton a des conséquences fiscales et réglementaires directes.
- La gestion de la clé privée déplace vers l'utilisateur final une responsabilité de sécurité normalement assumée par une institution financière, sans mécanisme de recours en cas de perte ou de vol — un point de fragilité majeur pour l'adoption grand public.
- La pertinence d'une solution blockchain doit être évaluée par rapport à un besoin réel de décentralisation, à l'aide d'une grille de décision explicite, et non adoptée par principe ou effet de mode.